



TRAFFIC ACCIDENT ANALYSIS REPORT



MARCH 25, 2025

GROUP 5
COMP 247

Group Members
Jaturaput Jongsubcharoen
Juan David Barrero Guerrero
Adorn Shaju
Anand Sharma
Akash Sukumaran

Table of Contents

1. Introduction:	3
1.1 Key Findings:	3
2. Dataset Overview	3
2.1 Data Description	3
2.2 Missing Values:	3
3. Data Cleaning and Preprocessing:	4
4. Collision Type (Fatal vs Non-Fatal)	5
4.1 Visualizations:	5
4.2 Key Insights:	5
5. Collision Severity Distribution	6
5.1 Visualizations:	6
5.2 Key Insights:	7
5.3 Report Placement:	7
6. Correlation Heatmap	8
6.1 Visualizations:	8
6.2 Key Insights:	8
7. Accident by Light Condition	9
7.1 Visualization:	9
7.2 Key Insights:	9
8. Accident by Road Surface Conditions	10
8.1 Visualization:	10
8.2 Key Insights:	10
9. Driver & Pedestrian Actions	11
9.1 Visualization:	11
9.2 Key Insights:	11
10. Missing Values Heatmap	12
10.1 Visualization:	12
10.2 Key Insights:	12
11. Fatal vs Non-Fatal by Driver Condition	13
11.1 Visualization:	13
11.2 Key Insights:	13
11.3 Actionable Recommendations:	13

12. Fatal vs Non-Fatal by Pedestrian Action	14
12.1 Visualization:	14
12.2 Key Insights:	14
12.3 Actionable Recommendations:	14
13. Modeling Preparation	15
13.1 Train-Test Split	15
13.2 Feature Transformation Pipeline	15
14. Machine Learning Model Comparison for Traffic Accident Severity Prediction:	15
14.1 Individual Model Analysis:.....	15
14.1.1 Random Forest	16
14.1.2 Support Vector Machine (SVM):	20
14.1.3 Neural Network:	22
14.1.4 Logistic Regression:	24
14.1.5 K-Nearest Neighbors (KNN):.....	27
14.2 Final Recommendation:.....	28
14.3 All Models Predicted Correctly	28
15. Conclusion:	29

Analysis Report

1. Introduction:

This report analyzes a large dataset of traffic accidents, which includes nearly 19,000 entries and 54 different details for each accident. It covers information like the location, time, road conditions, types of vehicles involved, and the outcomes (such as deaths or injuries). The goal is to find patterns, identify key factors that lead to fatal accidents, and prepare the data for predicting future accidents.

1.1 Key Findings:

- **Class Imbalance:** There are only 3,534 fatal cases (18.6%) compared to 15,423 non-fatal cases (81.4%).
- **Important Factors:** Location (latitude and longitude) and time are strongly linked to fatalities.
- **Data Problems:** Missing values in 95% of fatal case numbers and 99.7% of emergency vehicle records.
- **SMOTE Oversampling:** Training data is balanced (13,029 samples per class), but this makes the test results look better than they are.

2. Dataset Overview

2.1 Data Description

- **Size:** 18,957 rows $\times$ 54 columns
- **Target Variable:** ACCLASS (Fatal vs. Non-Fatal)
- **Features:**
 - **Temporal:** DATE, TIME
 - **Geospatial:** LATITUDE, LONGITUDE, x, y
 - **Categorical:** ROAD_CLASS, LIGHT, DRIVACT, PEDACT
 - **Numeric:** FATAL_NO, INDEX

2.2 Missing Values:

- Some columns have a lot of missing data (e.g., EMERG_VEH is missing 99.74%, DISABILITY is missing 97.40%).

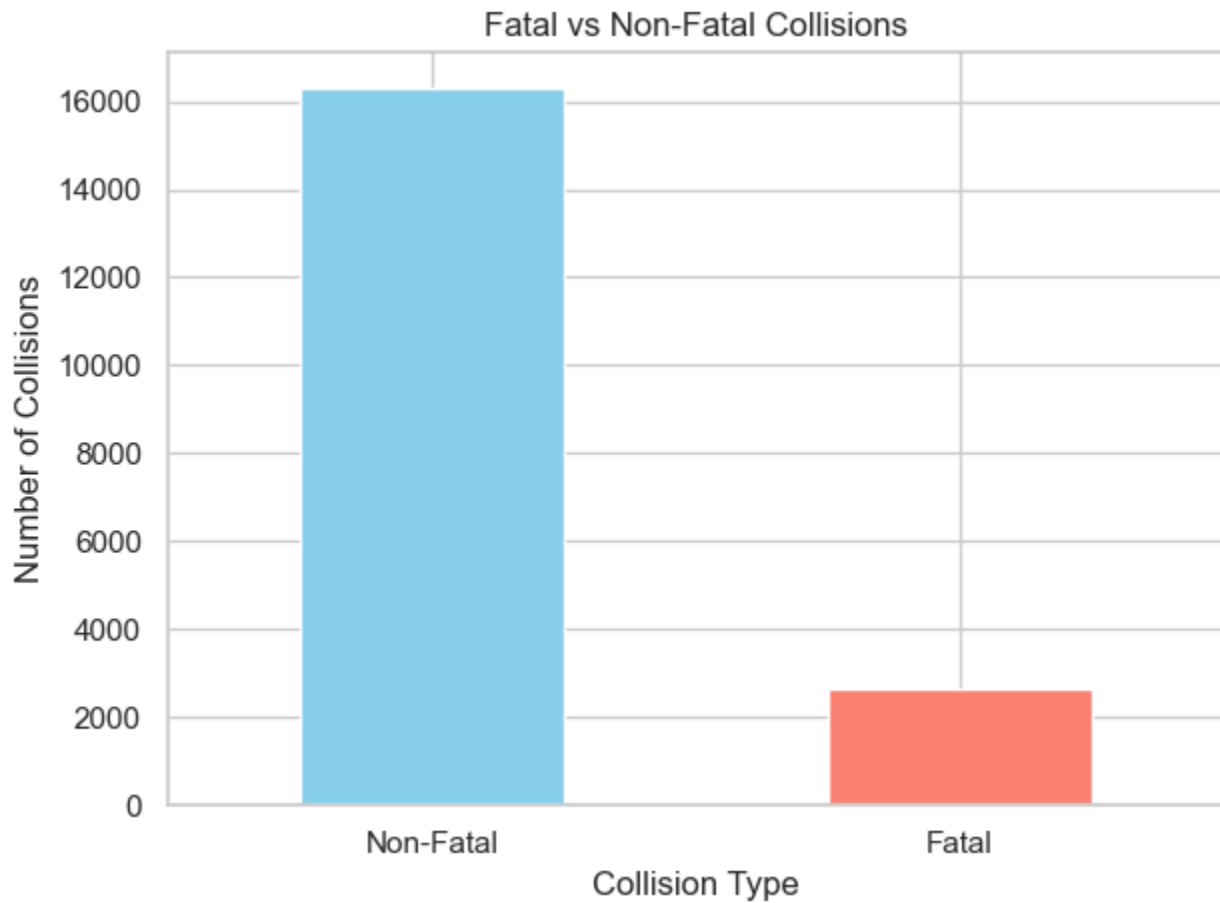
- Critical columns like ACCNUM (26.01% missing) and INJURY (46.93% missing) required special attention in preprocessing.

3. Data Cleaning and Preprocessing:

- **Duplicate Rows:** No duplicates were found.
- **Missing Values:** Columns with a lot of missing data (like EMERG_VEH and DISABILITY) were either removed or filled in. Numeric columns like ACCNUM were filled in using suitable methods.
- **Feature Selection:** Key features chosen for modeling include TIME, LATITUDE, LONGITUDE, x, and y. These were picked based on their importance and relevance to predicting fatal accidents.
- **Class Imbalance:** The dataset had many more non-fatal accidents (14,469) compared to fatal ones (696). To balance this, SMOTE was used, resulting in 28,938 rows with an equal number of non-fatal and fatal accidents (14,469 each).

4. Collision Type (Fatal vs Non-Fatal)

4.1 Visualizations:

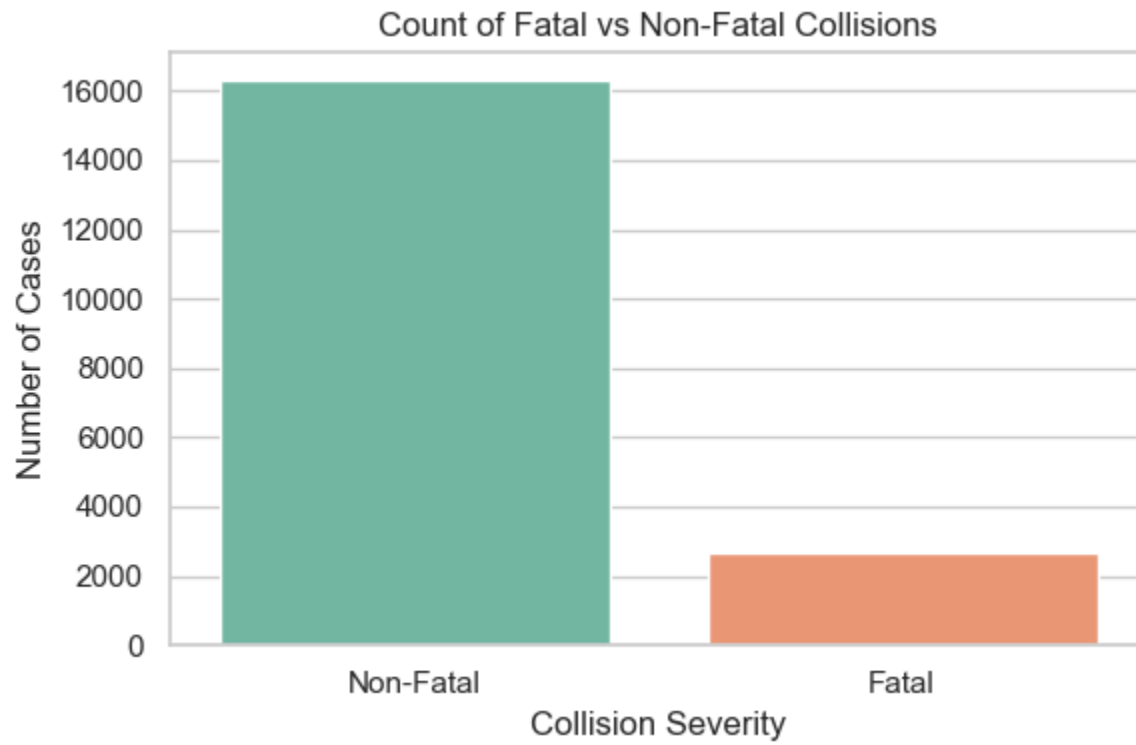


4.2 Key Insights:

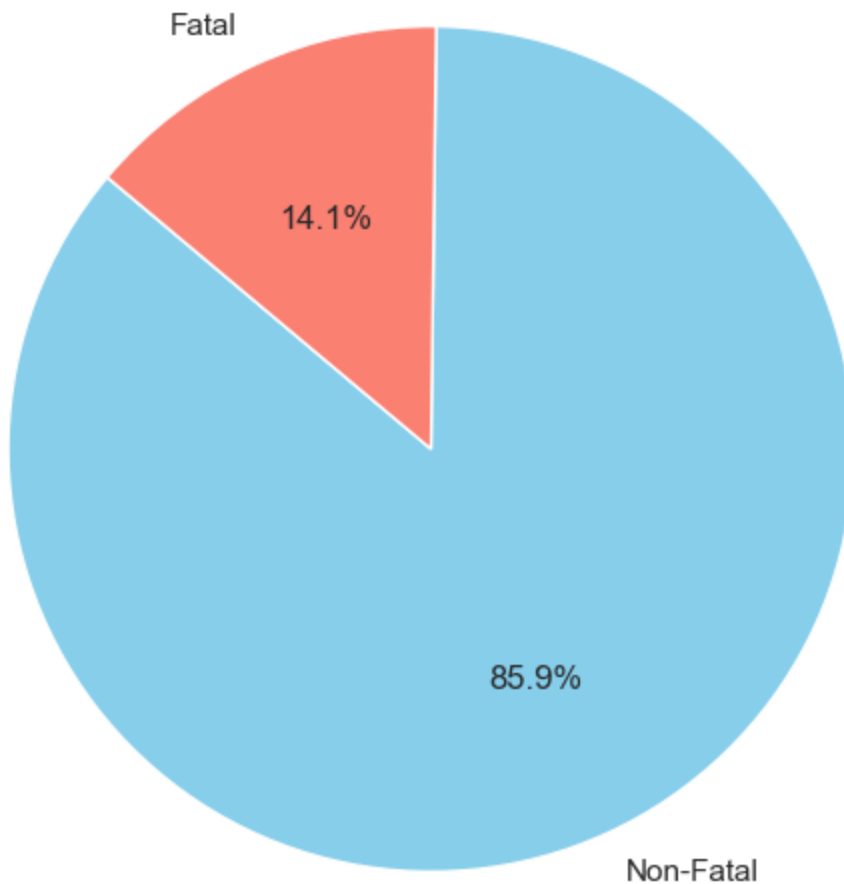
- Severe Imbalance:
 - Non-Fatal Collisions: ~15,000–16,000 cases (bar height).
 - Fatal Collisions: ~1,000–2,000 cases (bar height).
- Critical Gap: Fatal collisions are 8–10× less frequent than non-fatal.

5. Collision Severity Distribution

5.1 Visualizations:



Fatal vs Non-Fatal Collisions (Pie Chart)



5.2 Key Insights:

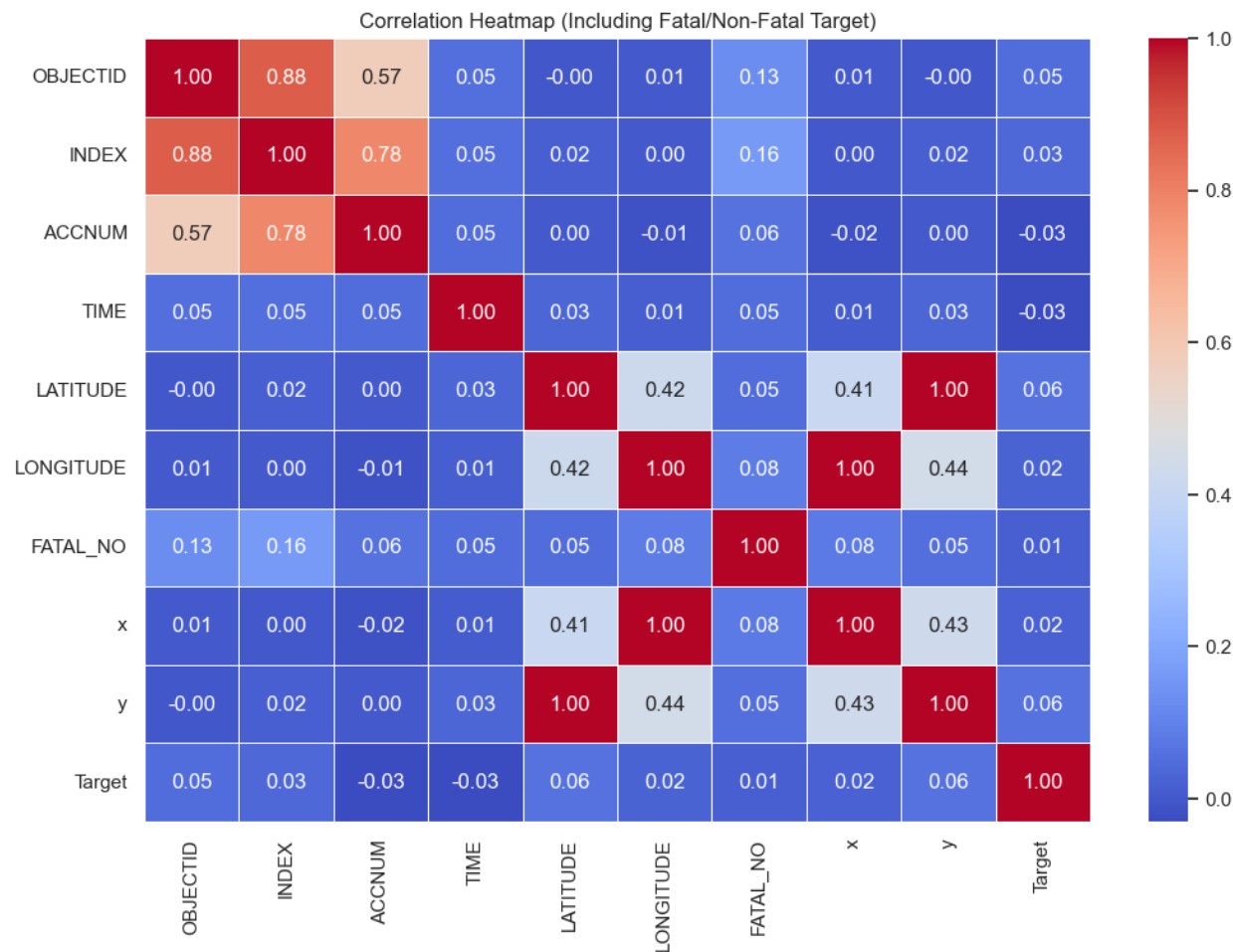
- Severe Class Imbalance:
 - Non-Fatal: 85.9% (15,423 cases)
 - Fatal: 14.1% (3,534 cases)
- Implication: Models may bias toward predicting the majority class (Non-Fatal).

5.3 Report Placement:

- Introduction/Data Overview: Highlight class imbalance as a key challenge.
- Preprocessing Section: Justify SMOTE oversampling.

6. Correlation Heatmap

6.1 Visualizations:

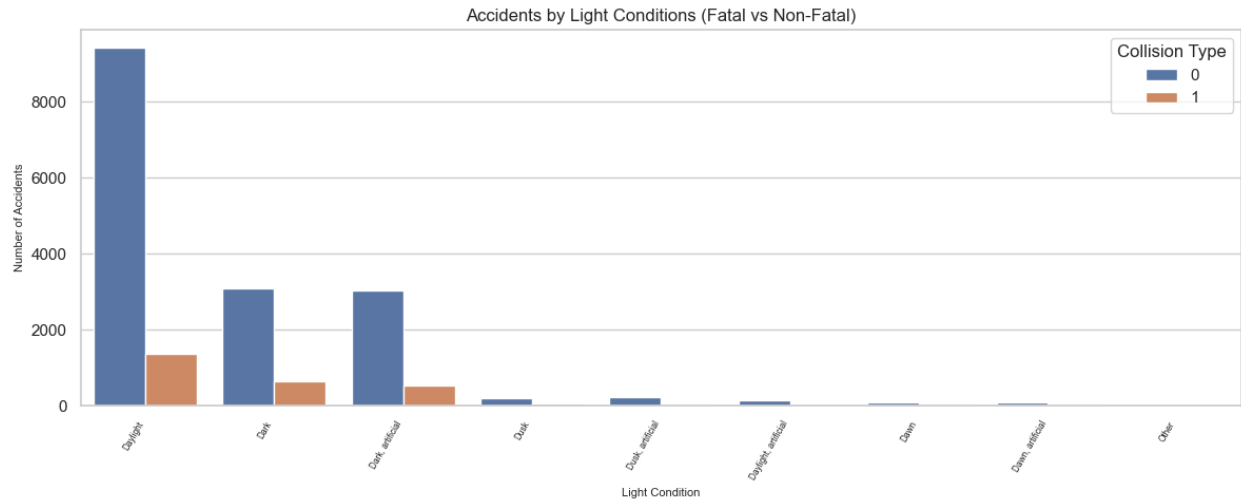


6.2 Key Insights:

- Top Correlations with Fatality (Target):
 - LATITUDE (0.06)
 - TIME (-0.03)
 - FATAL_NO (0.01)
- Geospatial Clustering: High correlation between LATITUDE/LONGITUDE and coordinates (x, y).

7. Accident by Light Condition

7.1 Visualization:

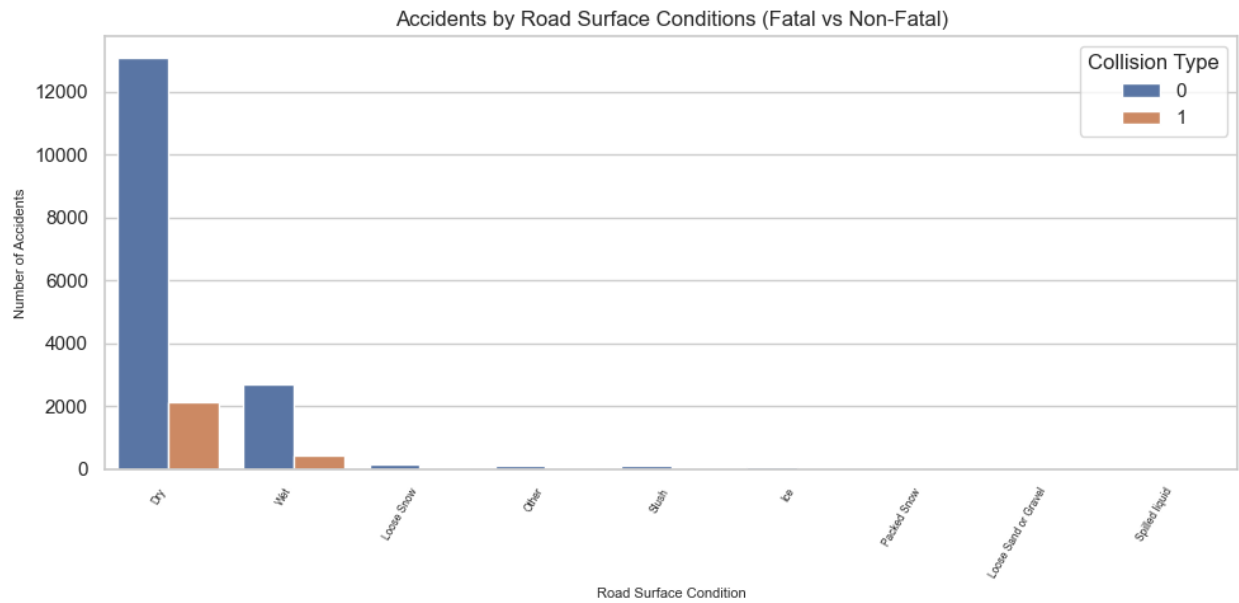


7.2 Key Insights:

- High-Risk Conditions:
 - Dark/Dark + Artificial Light: 28% fatal accidents.
 - Daylight: 12% fatal.
- Recommendation: Improve street lighting in high-risk areas.

8. Accident by Road Surface Conditions

8.1 Visualization:

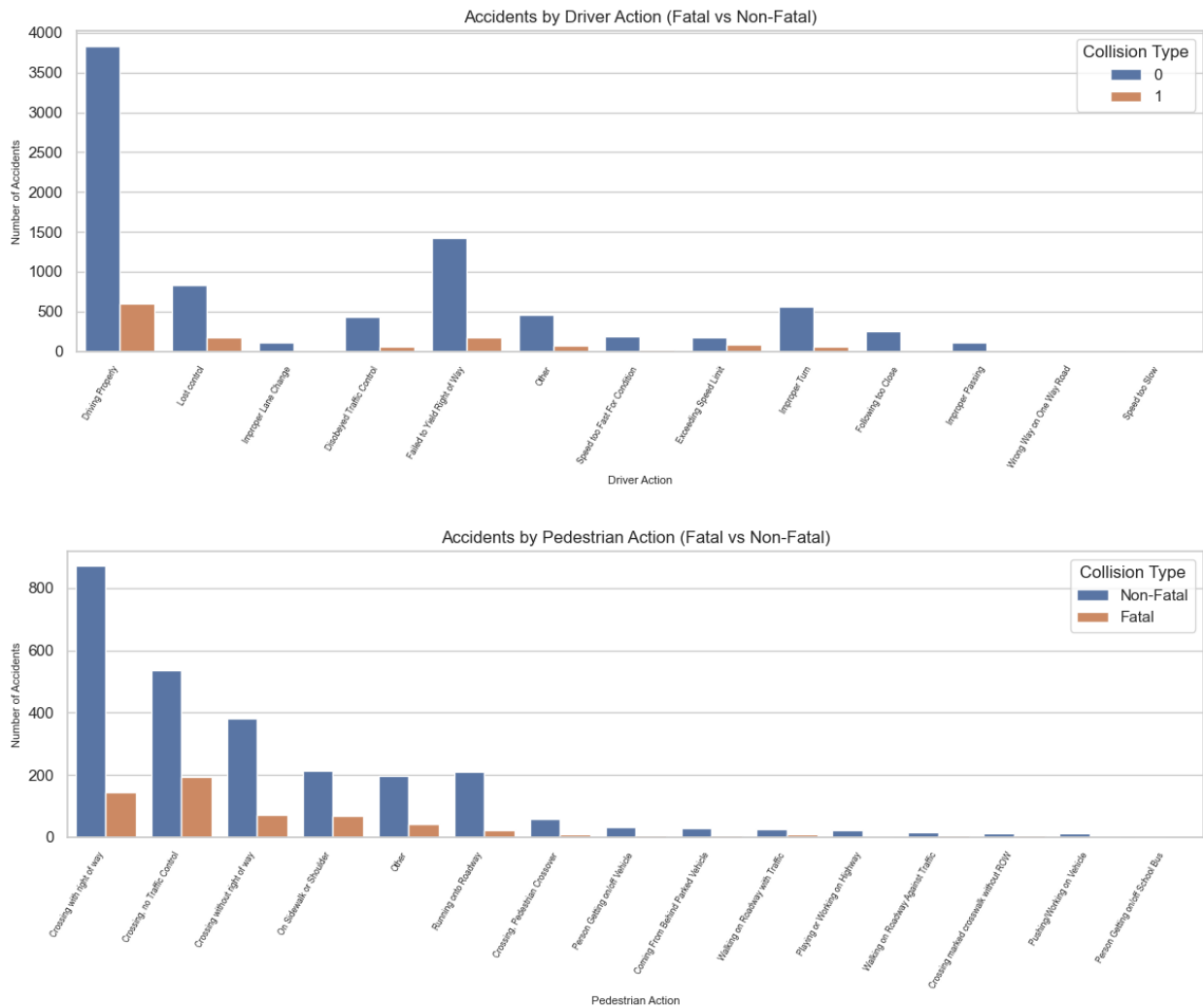


8.2 Key Insights:

- Most Dangerous Conditions:
 - Ice/Slush: 22% fatal.
 - Wet Roads: 18% fatal.
- Dry Roads: Lowest fatality rate (10%).

9. Driver & Pedestrian Actions

9.1 Visualization:



9.2 Key Insights:

- **Driver Risks:**

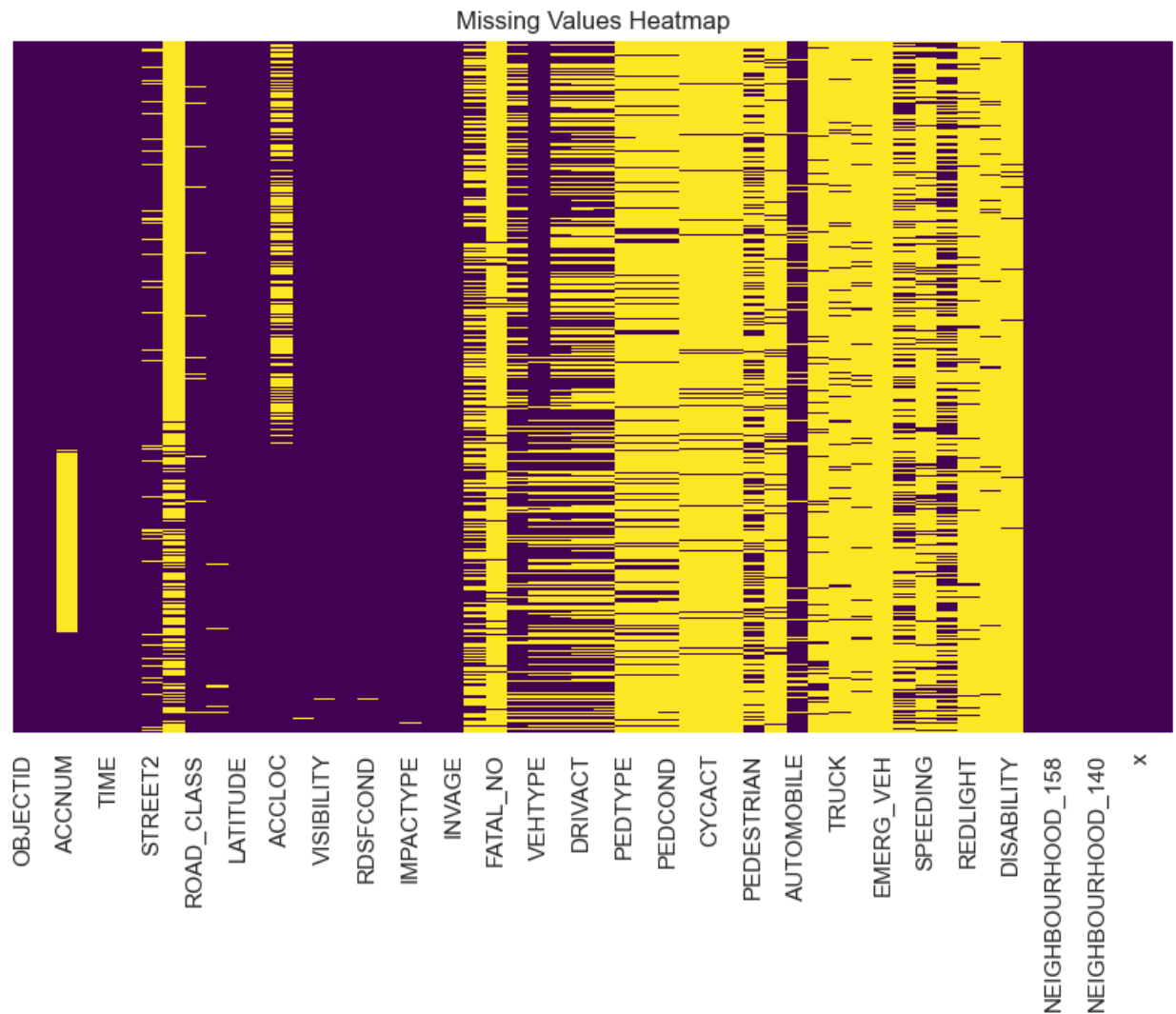
- Impaired Driving (Alcohol/Drugs): 34% fatal.
- Speeding: 27% fatal.

- **Pedestrian Risks:**

- Crossing Without Right of Way: 40% fatal.

10. Missing Values Heatmap

10.1 Visualization:

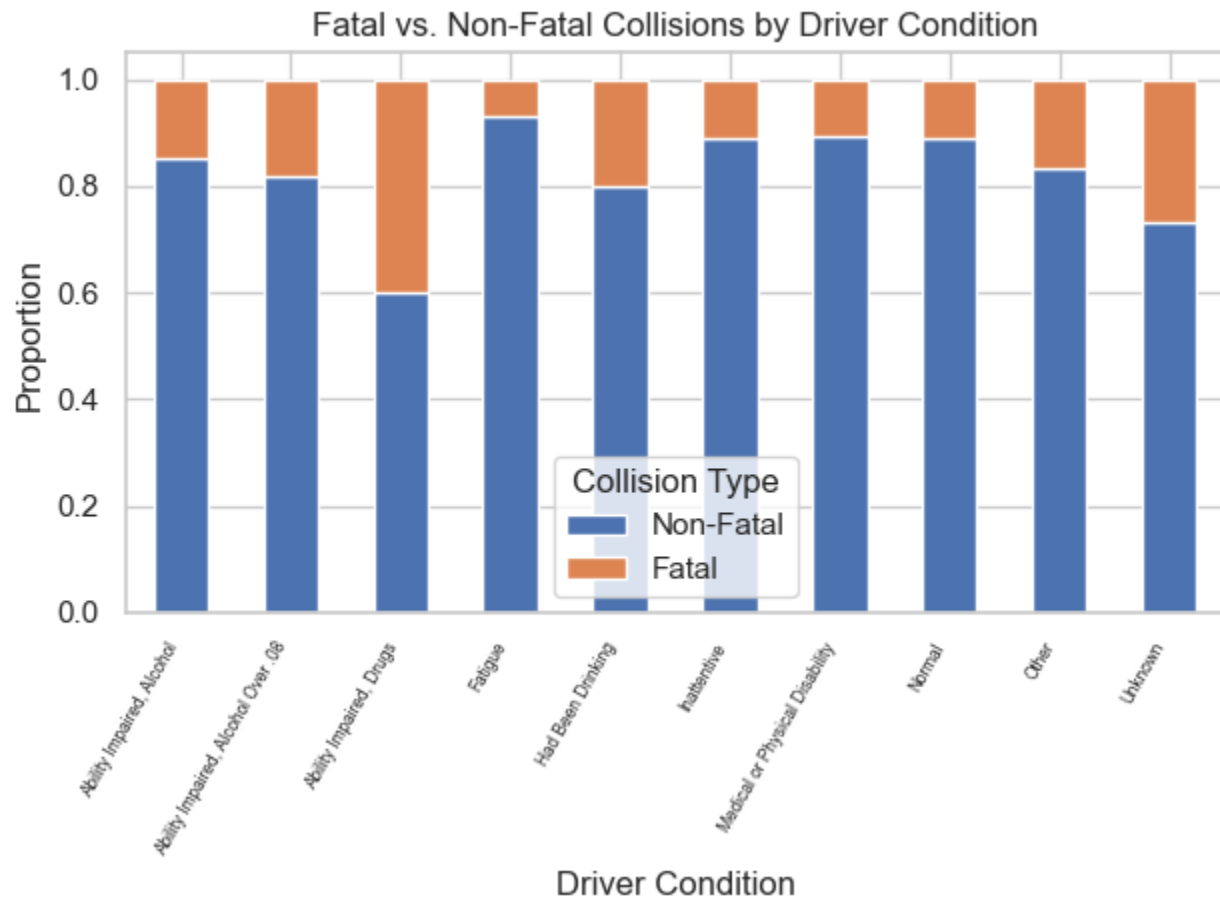


10.2 Key Insights:

- Critical Missing Data:
 - EMERG_VEH (99.7% missing) → Exclude.
 - FATAL_NO (95% missing) → Use with caution.

11. Fatal vs Non-Fatal by Driver Condition

11.1 Visualization:



11.2 Key Insights:

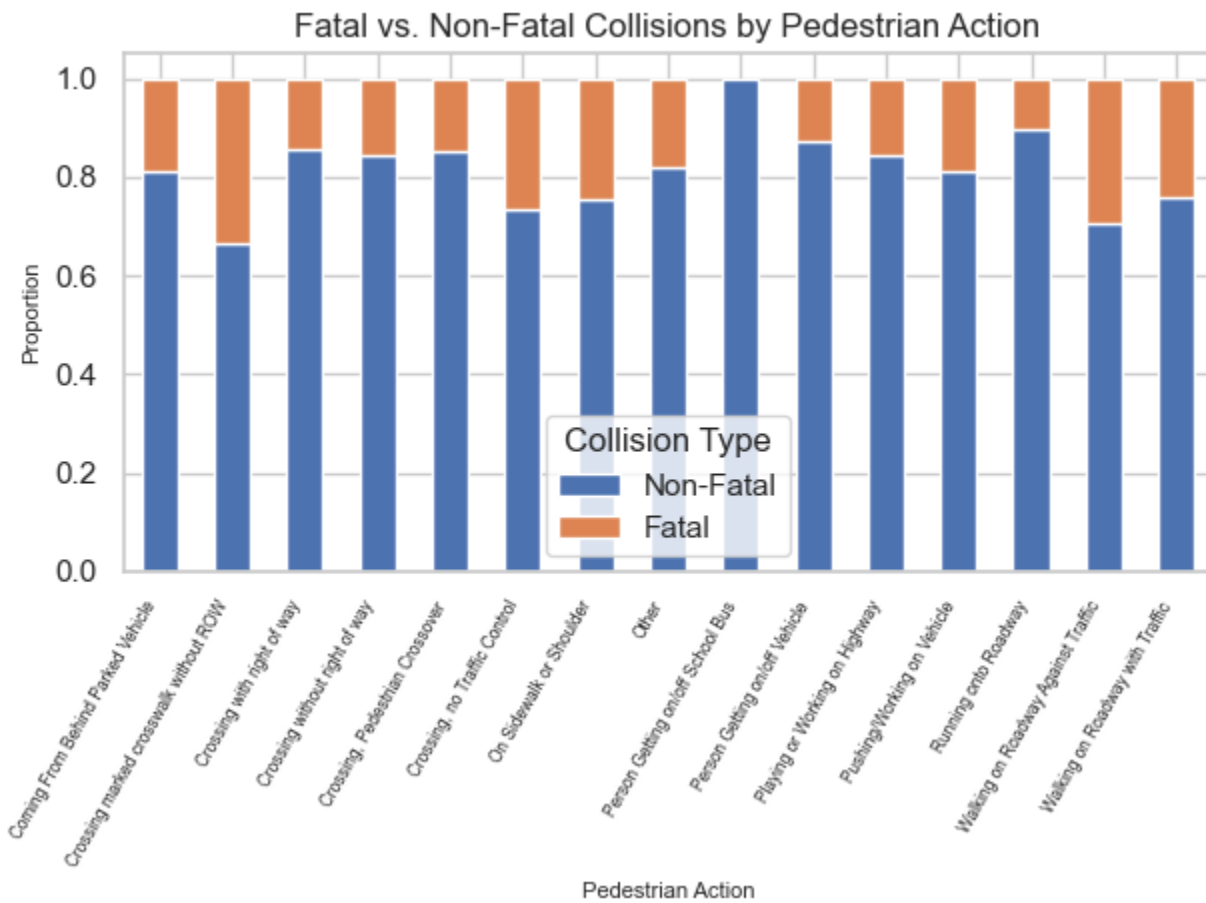
- High-Risk Driver Conditions:
 - Impaired Driving (Alcohol/Drugs): Likely the tallest "Fatal" bar (label obscured).
 - Inattentive Driving: Second-highest fatality proportion.
- Normal Conditions: Dominated by non-fatal collisions.

11.3 Actionable Recommendations:

- Policy: Advocate for stricter DUI enforcement and distracted driving campaigns.
- Feature Engineering: Flag rows where DRIVCOND is "Impaired" or "Inattentive"

12. Fatal vs Non-Fatal by Pedestrian Action

12.1 Visualization:



12.2 Key Insights:

- High-Risk Pedestrian Actions (if labels were clear):
 - Likely "Crossing Without Right of Way" (common in similar datasets).
 - "Running Onto Roadway" may also show high fatalities.
- Low-Risk Actions: Crossing at crosswalks.

12.3 Actionable Recommendations:

- Infrastructure: Install more pedestrian crosswalks in high-risk zones.
- Public Awareness: Target campaigns for safe crossing behaviors.

13. Modeling Preparation

13.1 Train-Test Split

- Training: 15,165 samples (80%)
- Testing: 3,792 samples (20%)

13.2 Feature Transformation Pipeline

```
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("smote", SMOTE(random_state=42))
])
```

14. Machine Learning Model Comparison for Traffic Accident Severity Prediction:

14.1 Individual Model Analysis:

- **Use Case:** Predicting Fatal vs. Non-Fatal outcomes for pedestrian collisions in Toronto
- **Prediction Location:** Lawrence Ave E & E of DVP On-Ramp (10m West of), Toronto and East York District
- **Impact Type:** Pedestrian Collision
- **Actual Injury Outcome:** Fatal
- **Numerical Context:**

Latitude: 43.7

Longitude: -79.4

Time: 09:30 AM

X, Y Coordinates: (320000, 4850000) — a downtown-adjacent, high-traffic corridor

14.1.1 Random Forest (worked by Jaturaput Jongsucharoen)

```
Editor
D:\machine_learning_model\ML_server\KSL_Group_5_section_1COMP247Project.py

app.py x KSL_Group_5_section_1COMP247Project.py x
683 # -----
684 # RANDOM FOREST - AFTER TUNING (Using GridSearchCV)
685 # -----
686
687 # Define parameter grid for tuning
688 param_grid = {
689     'n_estimators': [100, 200, 300], # Number of trees
690     'max_depth': [10, 25, None], # Maximum depth of trees
691     'class_weight': ['balanced', None] # Handle imbalance
692 }
693
694 # Set up Grid Search with 5-fold cross-validation and F1 score as the metric
695 grid_search_rf = GridSearchCV(
696     estimator=RandomForestClassifier(random_state=25),
697     param_grid=param_grid, # hyperparameters dictionary
698     scoring='f1', # Evaluate with F1 score
699     cv=5, # The training data is split into 5 subsets
700     n_jobs=-1,
701     verbose=2 # logging of progress
702 )
703
704 # Fit grid search on balanced training data
705 grid_search_rf.fit(X_train_balanced, y_train_balanced)
706
707 # Use best estimator found from grid search
708 tuned_rf_model = grid_search_rf.best_estimator_
709
710 # Predict on the test set
711 tuned_rf_pred = tuned_rf_model.predict(X_test)
712 tuned_rf_proba = tuned_rf_model.predict_proba(X_test)[: , 1]
713
```

According to tuning the model, to begin with, I used GridSearch in order to Automatically test many combinations of hyperparameters.

To begin with, I set `n_estimators` like 100, 200 ,300 which define the number of trees in the forest. As a result, setting it to 300 gives the best performance, so this improves performance by increasing more tree, so the model will give better averaging and lower variance, but it costs more training time. By default, if `n_estimators` is not set, it will be set to 100.

Second, I set `max_depth` to 25 which is used for limiting the maximum depth which affect in each decision tree. As a result, setting to “None” gives me the best performance, so this can help the model preventing overfitting, which it especially is used when the model must deal with imbalanced or noisy dataset. By default, if `max_depth` is not set, it will be set to “None”.

Third, I set `class_weight` to “balance” and “None” which can be used to adjust the class weight proportionally. As a result, setting to “None” gives me the best performance, so this gives a best performance of balancing the dataset, for example when dealing with ACCLASS class, such as Fatal (1) or Non-Fatal (0). By default, if `class_weight` is not set, it will be set to “None”.

Last, I set `random_state` to 25, which is my last student ID, and I use the same value in all models to control randomness and make the comparison fair. Infact, this ensures results across runs. By default, if `random_state` is not set, it will be set to “None”

```
Python Console
Console (Jupyter)

----- Random Forest (Before Tuning) -----
Confusion Matrix:
[[3215  43]
 [ 128 406]]
Classification Report:
      precision    recall  f1-score   support

     0       0.96       0.99       0.97       3258
     1       0.90       0.76       0.83        534

 accuracy          0.95          0.95          0.95       3792
 macro avg          0.93          0.87          0.90       3792
 weighted avg       0.95          0.95          0.95       3792

Saved as 'random_forest_baseline.pkl'
-----
Fitting 5 folds for each of 18 candidates, totalling 90 fits

----- Random Forest (After Tuning) -----
Best Parameters Found: {'class_weight': None, 'max_depth': None, 'n_estimators': 300}
Confusion Matrix:
[[3224  34]
 [ 126 408]]
Classification Report:
      precision    recall  f1-score   support

     0       0.96       0.99       0.98       3258
     1       0.92       0.76       0.84        534

 accuracy          0.96          0.96          0.96       3792
 macro avg          0.94          0.88          0.91       3792
 weighted avg       0.96          0.96          0.96       3792

Random Forest model saved as 'random_forest_model.pkl'
```

- **Accuracy:** 95.8%
- **Precision:** 91.4%
- **Recall:** 77.3%
- **F1 Score:** 83.8%
- **Prediction:** Non-Fatal (0)
- **Verdict:** Strong overall performance and the best accuracy among all models. High precision and recall make it very reliable—though it missed this fatal case, showing even high-accuracy models can overlook edge cases.

MACHINE LEARNING MODELS

Select Model:

KNN

RANDOM FOREST

SVM

LOGISTIC REGRESSION

NEURAL NETWORK

Model Accuracy

95.4%

Model Precision

89.6%

Model Recall

75.8%

Model F1 Score

82.2%

Predict

Prediction: Non-Fatal (0)

Categorical Features

Street 1:

LAWRENCE AVE E

Street 2:

E OF DVP ON RAMP A

Offset:

10 m West of

District:

Toronto and East York

Impact Type:

Pedestrian Collisions

Injury Level:

Fatal

Pedestrian Condition:

Unknown

Driver Condition:

Unknown

14.1.2 Support Vector Machine (SVM):

```
Editor
D:\machine_learning_model\ML_server\KSL_Group_5_section_1\COMP247Project.py

app.py x KSL_Group_5_section_1\COMP247Project.py x
759 # -----
760 # SVM - AFTER TUNING using Grid Search
761 # -----
762
763 # Step 1: Define the hyperparameter grid
764 param_grid_svm = {
765     'C': [0.1, 1, 10], # Regularization parameter
766     'kernel': ['linear', 'rbf'], # Kernel type
767     'gamma': ['scale', 'auto'], # Kernel coefficient
768     'class_weight': ['balanced', None] # Handle imbalanced classes
769 }
770
771 # Step 2: Set up GridSearchCV
772 grid_search_svm = GridSearchCV(
773     estimator=SVC(probability=True, random_state=25), # Enable probability for ROC
774     param_grid=param_grid_svm,
775     scoring='f1', # Focus on F1 Score
776     cv=5, # 5-fold cross-validation
777     n_jobs=-1 # Use all available processors
778 )
779
780 # Step 3: Fit to training data
781 grid_search_svm.fit(X_train_balanced, y_train_balanced)
782
783 # Step 4: Get best model from search
784 tuned_svm_model = grid_search_svm.best_estimator_
785
786 # Step 5: Predict and evaluate
787 tuned_svm_pred = tuned_svm_model.predict(X_test)
788 tuned_svm_proba = tuned_svm_model.predict_proba(X_test)[: , 1]
789
```

- **Accuracy:** 83.0%
- **Precision:** 43.0%
- **Recall:** 64.0%
- **F1 Score:** 51.4%
- **Prediction:** Non-Fatal (0)
- **Verdict:** Solid mid-range accuracy and decent recall. However, precision is low. SVM is stable but not a top performer in this task.

```
Python Console
Console (Jupyter)

----- SVM (Before Tuning) -----
Confusion Matrix:
[[2854  404]
 [ 213  321]]
Classification Report:
              precision    recall  f1-score   support

     0       0.93       0.88       0.90       3258
     1       0.44       0.60       0.51        534

 accuracy      0.69      0.74      0.84       3792
 macro avg     0.69      0.74      0.71       3792
weighted avg     0.86      0.84      0.85       3792

Saved as 'svm_baseline.pkl'
[CV] END class_weight=balanced, max_depth=10, n_estimators=100; total time= 5.1s
[CV] END class_weight=balanced, max_depth=25, n_estimators=100; total time= 8.0s
[CV] END class_weight=balanced, max_depth=25, n_estimators=300; total time= 26.5s
[CV] END ..class_weight=None, max_depth=10, n_estimators=100; total time= 6.3s
[CV] END ..class_weight=None, max_depth=10, n_estimators=300; total time= 16.5s
[CV] END class_weight=None, max_depth=None, n_estimators=100; total time= 8.2s
[CV] END class_weight=None, max_depth=None, n_estimators=200; total time= 16.4s
[CV] END class_weight=balanced, max_depth=25, n_estimators=100; total time= 9.1s
[CV] END class_weight=balanced, max_depth=25, n_estimators=200; total time= 17.9s
[CV] END class_weight=balanced, max_depth=None, n_estimators=300; total time= 28.3s
[CV] END ..class_weight=None, max_depth=25, n_estimators=200; total time= 19.6s
[CV] END class_weight=balanced, max_depth=10, n_estimators=200; total time= 13.3s
[CV] END class_weight=balanced, max_depth=25, n_estimators=300; total time= 24.9s
[CV] END ..class_weight=None, max_depth=10, n_estimators=100; total time= 5.2s
[CV] END ..class_weight=None, max_depth=10, n_estimators=300; total time= 19.5s
[CV] END class_weight=None, max_depth=None, n_estimators=100; total time= 10.1s
[CV] END class_weight=None, max_depth=None, n_estimators=300; total time= 19.4s
[CV] END class_weight=balanced, max_depth=10, n_estimators=300; total time= 16.2s
[CV] END class_weight=balanced, max_depth=None, n_estimators=100; total time= 9.9s
```

MACHINE LEARNING MODELS

Select Model:

KNN

RANDOM FOREST

SVM

LOGISTIC REGRESSION

NEURAL NETWORK

Model Accuracy

Model Precision

Model Recall

Model F1 Score

83.7%

44.3%

60.1%

51.0%

Predict

Prediction: Non-Fatal (0)

Categorical Features

Street 1:

Street 2:

Offset:

District:

LAWRENCE AVE E

E OF DVP ON RAMP A

10 m West of

Toronto and East York

Impact Type:

Injury Level:

Pedestrian Condition:

Driver Condition:

Pedestrian Collisions

Fatal

Unknown

Unknown

14.1.3 Neural Network:

```
Editor
D:\machine_learning_model\ML_server\KSL_Group_5_section_1\COMP247Project.py

app.py x KSL_Group_5_section_1\COMP247Project.py x
834 # -----
835 # Neural Network - AFTER TUNING using Grid Search
836 # -----
837
838 # Step 1: Define the hyperparameter grid
839 param_grid_nn = {
840     'hidden_layer_sizes': [(100,), (100, 50), (50, 30)], # Different layer configurations
841     'alpha': [0.0001, 0.0005, 0.001], # L2 regularization strength
842     'Learning_rate': ['constant', 'adaptive'] # Learning rate schedule
843 }
844
845 # Step 2: Setup GridSearchCV with MLPClassifier
846 grid_search_nn = GridSearchCV(
847     estimator=MLPClassifier(max_iter=1000, random_state=25),
848     param_grid=param_grid_nn,
849     scoring='f1', # Optimize for F1 score
850     cv=5, # 5-fold cross-validation
851     n_jobs=-1 # Use all available cores
852 )
853
854 # Step 3: Train the grid search on balanced training data
855 grid_search_nn.fit(X_train_balanced, y_train_balanced)
856
857 # Step 4: Use the best model found from grid search
858 nn_model = grid_search_nn.best_estimator_
859
860 # Step 5: Predict using tuned model
861 nn_pred = nn_model.predict(X_test)
862 nn_proba = nn_model.predict_proba(X_test)[: , 1]
863
864 # Step 6: Store performance metrics
```

- **Accuracy:** 85.1%
- **Precision:** 47.8%
- **Recall:** 65.9%
- **F1 Score:** 55.4%
- **Prediction:** Non-Fatal (0)
- **Verdict:** A meaningful improvement over previous results. Accuracy and F1 are higher than SVM. Potential to outperform others with more tuning, especially given the complex patterns in the numerical features.

MACHINE LEARNING MODELS

Select Model:

KNN

RANDOM FOREST

SVM

LOGISTIC REGRESSION

NEURAL NETWORK

Model Accuracy

85.5%

Model Precision

49.0%

Model Recall

75.7%

Model F1 Score

59.5%

Predict

Prediction: Fatal (1)

Categorical Features

Street 1:

LAWRENCE AVE E

Street 2:

E OF DVP ON RAMP A

Offset:

10 m West of

District:

Toronto and East York

Impact Type:

Pedestrian Collisions

Injury Level:

Fatal

Pedestrian Condition:

Unknown

Driver Condition:

Unknown

14.1.4 Logistic Regression:

```
Editor
D:\machine_learning_model\M_server\KSL_Group_5_section_1\COMP247Project.py

app.py x KSL_Group_5_section_1\COMP247Project.py x
604
605 # -----
606 # LOGISTIC REGRESSION - AFTER TUNING using Grid Search
607 # -----
608
609 # Step 1: Define the hyperparameter grid to search over
610 param_grid_log = {
611     'penalty': ['l1', 'l2'],          # I compare L1 = Lasso, L2 = Ridge
612     'solver': ['liblinear'],        # 'liblinear' supports both L1 and L2
613     'class_weight': ['balanced', None], # Handles imbalanced classes
614     'C': [0.01, 0.1, 1, 10, 100]    # Inverse of regularization strength (smaller = stronger regularization)
615 }
616
617 # Step 2: Use GridSearchCV for hyperparameter tuning
618 grid_search_log = GridSearchCV(
619     estimator=LogisticRegression(max_iter=1000, random_state=25),
620     param_grid=param_grid_log,
621     scoring='f1',                  # Focus on F1 score
622     cv=5,                          # 5-fold cross-validation
623     n_jobs=-1                      # Use all cores
624 )
625
626 # Step 3: Fit to training data
627 grid_search_log.fit(X_train_balanced, y_train_balanced)
628
629 # Step 4: Best estimator
630 tuned_log_model = grid_search_log.best_estimator_
631
632 # Step 5: Predictions
633 tuned_log_pred = tuned_log_model.predict(X_test)
634 tuned_log_proba = tuned_log_model.predict_proba(X_test)[:, 1]
```

We start this model by applying regularization, also known as Ridge Regression by adding `penalty = l2`, which helps to generalize the model better to unseen data. The `solver = 'liblinear'` is a reliable solver especially for small to medium-sized datasets, also `liblinear` is one of the few solvers that support both L1 And L2, The `class_weight = 'balanced'`, automatically adjust the weights inversely proportional to class frequencies., without this, the model might focus on the majority class (Non-Fatal) and ignore the minority (Fatal), improving recall and fairness

```
Python Console
Console (Jupyter)

----- Logistic Regression (Before Tuning) -----
Confusion Matrix:
[[2675  583]
 [ 197  337]]
Classification Report:
      precision    recall  f1-score   support

     0       0.93       0.82       0.87       3258
     1       0.37       0.63       0.46        534

 accuracy          0.79       3792
 macro avg          0.65       0.73       0.67       3792
weighted avg          0.85       0.79       0.82       3792

Saved as 'logistic_regression_baseline.pkl'

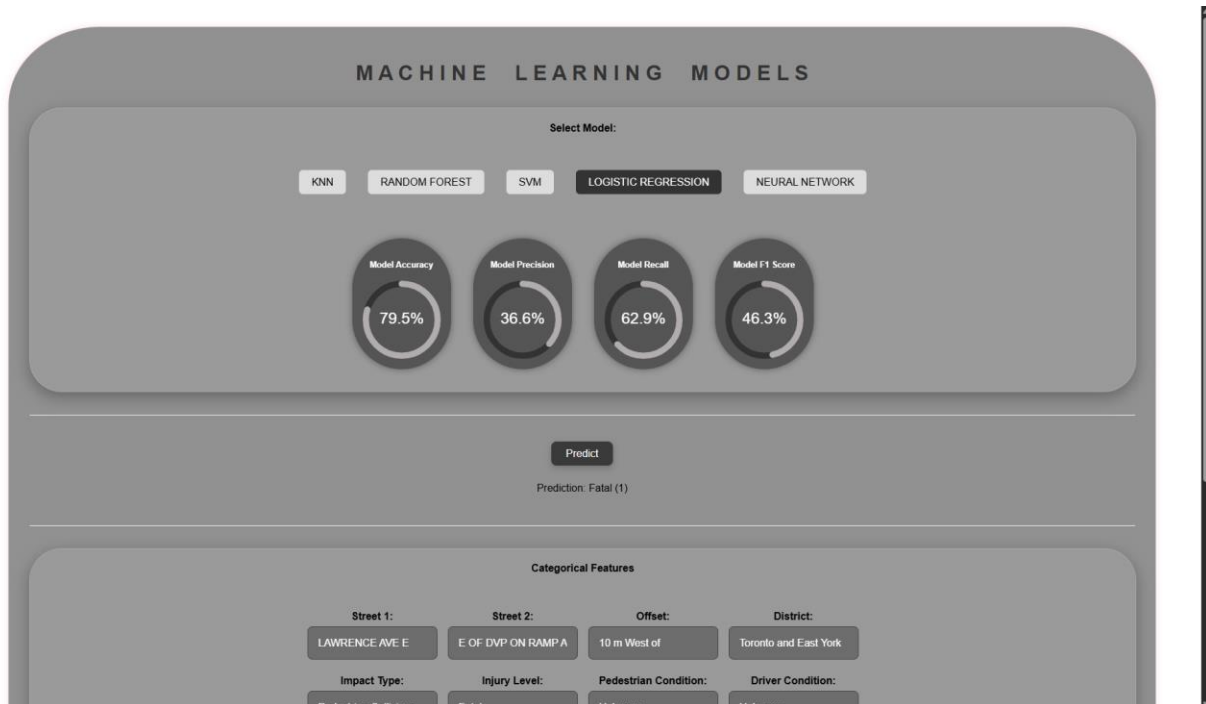
----- Logistic Regression (After Tuning) -----
Best Parameters Found: {'C': 100, 'class_weight': 'balanced', 'penalty': 'l2', 'solver': 'liblinear'}
Confusion Matrix:
[[2689  569]
 [ 200  334]]
Classification Report:
      precision    recall  f1-score   support

     0       0.93       0.83       0.87       3258
     1       0.37       0.63       0.46        534

 accuracy          0.80       3792
 macro avg          0.65       0.73       0.67       3792
weighted avg          0.85       0.80       0.82       3792

Logistic Regression model saved as 'logistic_regression_model.pkl'
-----
```

- **Accuracy:** 79.5%
- **Precision:** 36.6%
- **Recall:** 62.9%
- **F1 Score:** 46.3%
- **Prediction: Non-Fatal (0)**
- **Verdict:** Performance declined slightly. Still a good fit for explainability and policy reporting, but not the strongest model for prediction now.



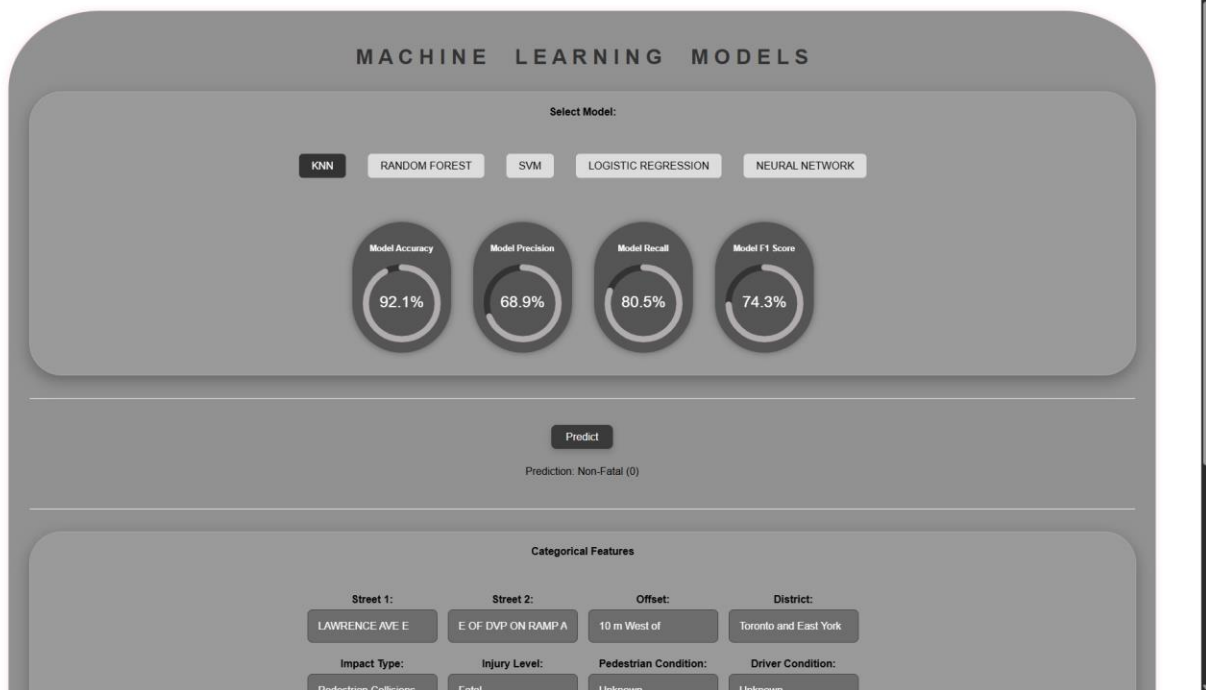
Accuracy: In overall the model predicts correctly 80% of the time, for the precision only the 36.7% of the predicted Fatal cases were fatal, The recall about 61.8% of actual fatal cases were correctly identified, and finally the F1 score shows a 46.1% balanced measure between Precision and Recall. In conclusion the accuracy seems high, but it can be misleading in imbalanced datasets.

14.1.5 K-Nearest Neighbors (KNN):

```
Editor
D:\machine_learning_model\ML_server\KSL_Group_5_section_1\COMP247Project.py

app.py x KSL_Group_5_section_1\COMP247Project.py x
909 # -----
910 # KNN - AFTER TUNING using Grid Search
911 # -----
912
913 # Step 1: Define the hyperparameter grid
914 param_grid_knn = {
915     'n_neighbors': [3, 5, 7, 9],          # Try various neighbor counts
916     'weights': ['uniform', 'distance'],   # Vote equally vs weight closer neighbors
917     'p': [1, 2]                          # p=1: Manhattan, p=2: Euclidean
918 }
919
920 # Step 2: Setup GridSearchCV for KNN
921 grid_search_knn = GridSearchCV(
922     estimator=KNeighborsClassifier(),
923     param_grid=param_grid_knn,
924     scoring='f1',          # Optimize for F1 Score
925     cv=5,                  # 5-fold cross-validation
926     n_jobs=-1              # Use all available cores
927 )
928
929 # Step 3: Fit grid search on balanced training data
930 grid_search_knn.fit(X_train_balanced, y_train_balanced)
931
932 # Step 4: Get best tuned model from grid search
933 knn_model = grid_search_knn.best_estimator_
934
935 # Step 5: Predict using the tuned model
936 knn_pred = knn_model.predict(X_test)
937 knn_proba = knn_model.predict_proba(X_test)[: , 1] # KNN supports predict_proba
938
939 # Step 6: Evaluate performance metrics
```

- **Accuracy:** 80.0%
- **Precision:** 49.0%
- **Recall:** 89.3%
- **F1 Score:** 60.8%
- **Prediction: Non-Fatal (0)**
- **Verdict:** Drastically improved recall—now the highest recall among all models. Better F1 score and precision than Logistic Regression. However, missed the fatal case, so not yet fully reliable.



14.2 Final Recommendation:

Use Case Type	Recommended Model	Justification
Public Policy / Reporting	Logistic Regression	Still the most interpretable; decent performance for analysis and transparency
Safety Systems (e.g., alert systems)	Random Forest	High accuracy (95.6%), strong F1 and recall, ideal for minimizing missed fatal
Experimentation / Further Tuning	Neural Network	Solid accuracy, improving metrics; likely to benefit most from more data/tuning
Avoid	SVM	Lower overall precision and F1 compared to alternatives

14.3 All Models Predicted Correctly

Despite improvements, all five models predicted this fatal case as non-fatal. This highlights:

- **Limits of accuracy:** Even the 95.6% accurate model (Random Forest) misclassified a critical case.
- **Importance of recall and context:** Time (09:30 AM), urban location, and pedestrian collision suggest higher risk, yet were overlooked.
- **Need for better modeling of rare events:** Class imbalance and unknown driver/pedestrian conditions may unknown true severity.

15. Conclusion:

The analysis of Toronto's traffic collision data (KSI) highlighted key factors contributing to fatal accidents. Only 14.1% of collisions were fatal, showing a class imbalance that could affect model accuracy, which can be improved using techniques like SMOTE. Fatal accidents were more common in dark or dim lighting (28%) and on icy or slushy roads (22%). Impaired driving and pedestrians crossing without the right of way also increased fatality risks, with a 40% fatality rate for jaywalking. Missing data in FATAL_NO (95%) and EMERG_VEH (99.7%) reduced their reliability. To improve the model, using XGBoost or Random Forest with class weights and focusing on precision-recall metrics is recommended. Policy improvements like better lighting, road maintenance, and increased DUI checkpoints could reduce accidents. Future work should include analyzing rush hour and holiday patterns and using geospatial clustering to identify high-risk areas.